

Practice 12 详解答案

A. 单项选择题

1. **B**: 类把相关的数据和操作这些数据的行为组合在一起。
2. **C**: `struct` 的成员默认是 `public`。
3. **C**: `class` 的成员默认是 `private`。
4. **B**: 对象是由类创建出的具体实例。
5. **B**: `Note myNote;` 创建一个名为 `myNote` 的 `Note` 对象。
6. **D**: `public` 成员可在类外访问。
7. **A**: 封装隐藏实现细节, 并通过接口保护和管理内部数据。
8. **C**: 笔记标题是对象的内部状态, 通常应为 `private`。
9. **B**: `Note` 类把笔记数据及其操作放在同一模型中。
10. **B**: 从组织函数进一步转为使用对象建模现实实体。

B. 填空题

1. 类 (`class`)
2. 对象 (实例)
3. `private` (私有)
4. 封装 (`encapsulation`)
5. 函数 (成员函数)

C. 判断题

1. **对**: 类可定义数据成员和成员函数。
2. **错**: 对象有相同的成员结构, 但每个对象通常拥有自己独立的数据值。
3. **错**: 两者都能有成员变量和成员函数, 关键区别是默认访问权限不同。
4. **对**: 类外代码不能直接读取或修改 `private` 成员。
5. **对**: 每个对象都有其非静态数据成员的一份数据。
6. **对**: 封装通过公开接口隐藏内部实现。
7. **对**: 这是一个名为 `Note` 的空类定义。
8. **对**: 同一个类的成员函数可访问该类对象的私有成员。
9. **对**: 类将相关状态和行为集中, 能减少大型程序的混乱。
10. **错**: 只要公开接口不变, 内部实现可以改变而不影响调用代码。

D. 代码阅读

1. 定义了什么

```
class Note
{
public:
    string title;
};
```

定义了一个 `Note` 类。它有一个公开的字符串数据成员 `title`；还没有创建任何 `Note` 对象。

2. 创建了什么

```
Note myNote;
```

创建了一个 `Note` 类的对象（实例），对象名称为 `myNote`。

3. 输出

题面中的 `intmain()` 应写为 `int main()`。修正后输出：

```
C++
```

`n` 是一个 `Note` 对象，`n.title` 使用类内给出的默认初始值 `"C++"`。

4. 代码问题

同样，`intmain()` 应写为 `int main()`。更重要的问题是：`title` 是 `private`，类外的 `main()` 不能执行 `n.title = "Study"`；。应提供公开成员函数：

```
class Note
{
private:
    string title;

public:
    void setTitle(const string& newTitle)
    {
        title = newTitle;
    }
};
```

调用： `n.setTitle("Study");`。

5. 将 `display()` 放进类的优势

`display()` 可以直接访问 `Note` 的私有数据；显示格式和笔记数据放在同一个抽象中，调用者只需写 `note.display()`，不必知道内部如何存储标题和内容。这提高了封装性、可维护性和复用性。

E. 简答题

1. 类与对象的区别

类是创建对象的蓝图，规定对象有哪些数据和行为；对象是类的具体实例。例如 `class Note` 描述笔记应有标题、内容和显示功能，`Note myNote;` 是一条实际笔记。多个 `Note` 对象共享类的定义，但分别保存自己的标题和内容。

2. 为什么封装是 OOP 的基本原则

封装将数据和操作数据的函数放在一起，并限制外部直接修改内部状态。类通过公开接口，例如 `setTitle()`，控制如何修改数据。这样可以集中校验规则，减少错误，并允许在不改变外部调用方式的前提下调整内部实现。

3. `Note` 类为何优于无关变量

独立的 `title`、`content` 和函数容易在多条笔记时混淆或传错参数。`Note` 对象把一条笔记的完整状态和操作集中起来，`note.save()`、`note.display()` 的归属明确；创建多条笔记时只需创建多个对象。

F. 编程题

1. 定义第一个类

```
#include <iostream>
#include <string>

using namespace std;

class Student
{
public:
    string name;
    int age;
};

int main()
{
    Student student;
    student.name = "Alice";
    student.age = 20;

    cout << "Name: " << student.name << '\n';
    cout << "Age: " << student.age << '\n';
}
```

2. 简单的 Note 类

```
#include <iostream>
#include <string>

using namespace std;

class Note
{
public:
    string title;
    string content;
};

int main()
{
    Note note;

    cout << "Enter title: ";
    getline(cin, note.title);
    cout << "Enter content: ";
    getline(cin, note.content);

    cout << "Title: " << note.title << '\n';
    cout << "Content: " << note.content << '\n';
}
```

3. 添加成员函数

```
#include <iostream>
#include <string>

using namespace std;

class Note
{
public:
    string title;
    string content;

    void display() const
    {
        cout << "==== NOTE =====\n";
        cout << "Title: " << title << '\n';
        cout << "Content: " << content << '\n';
        cout << "===== \n";
    }
};

int main()
{
    Note note;
    note.title = "Learning C++";
    note.content = "Practice every day.";
    note.display();
}
```

`display() const` 表示该成员函数只读对象，不会修改标题或内容。

4. 练习封装

```
#include <iostream>
#include <string>

using namespace std;

class Note
{
private:
    string title;

public:
    void setTitle(const string& newTitle)
    {
        title = newTitle;
    }

    string getTitle() const
    {
        return title;
    }
};

int main()
{
    Note note;
    note.setTitle("Study Classes");
    cout << "Title: " << note.getTitle() << '\n';
}
```

title 不能被 main() 直接访问，必须经由公开的 getter 和 setter。

5. CLI Notes OOP (挑战)

```
#include <fstream>
#include <iostream>
#include <limits>
#include <string>

using namespace std;

class Note
{
private:
    string title;
    string content;
    string tag;

public:
    void create()
    {
        cout << "Enter title: ";
        getline(cin, title);
        cout << "Enter content: ";
        getline(cin, content);
        cout << "Enter tag: ";
        getline(cin, tag);
    }

    void display() const
    {
        if (title.empty() && content.empty()) {
            cout << "No note available.\n";
            return;
        }

        cout << "==== NOTE =====\n";
        cout << "Title: " << title << '\n';
        cout << "Content: " << content << '\n';
        cout << "Tag: " << tag << '\n';
        cout << "=====\\n";
    }

    void save() const
    {
        ofstream file("notes.txt");
        if (!file) {
            cout << "Could not open notes.txt.\n";
        }
    }
}
```



```

        return;
    }

    file << title << '\n' << content << '\n' << tag << '\n';
    cout << "Note saved.\n";
}

void load()
{
    ifstream file("notes.txt");
    if (!file) {
        cout << "No saved note was found.\n";
        return;
    }

    getline(file, title);
    getline(file, content);
    getline(file, tag);
    cout << "Note loaded.\n";
}

};

int main()
{
    Note note;
    int choice = 0;

    while (choice != 5) {
        cout << "\n1. Create\n2. Display\n3. Save\n4. Load\n5. Exit\n";
        cout << "Choose: ";

        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Please enter a number.\n";
            continue;
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        switch (choice) {
            case 1: note.create(); break;
            case 2: note.display(); break;
            case 3: note.save(); break;
            case 4: note.load(); break;
            case 5: cout << "Goodbye.\n"; break;
            default: cout << "Invalid choice.\n";

```

```
    }  
  }  
}
```

这里 `Note` 对象拥有自己的数据，`main()` 只调用对象的公开操作，不需要维护全局标题、内容或标签变量。

G. 设计与思考挑战

设计 B 更符合 OOP，因为它将一条笔记的状态（`title`、`content`）与行为（`save()`、`display()`）封装进同一个 `Note` 对象。数据是私有的，外部代码只能使用公开接口；类可在内部改变保存或显示方式，而不必让所有调用代码一起修改。设计 A 的变量和函数没有明确归属，多条笔记时更容易混用数据。